

CTFL v4.0 Key Points

ISTQB Foundation Level syllabus concepts mapped to the QA Atelier project

Author	Tomas Xavier Santos
Project	QA Atelier (tom-xs/qa-atelier)
AUT	Cypress Real World App
Date	2026-07-30
Version	1.0

Contents

1. How to Use This Document	3
2. Chapter 1 — Fundamentals of Testing	3
3. Chapter 2 — Testing Throughout the SDLC	4
4. Chapter 3 — Static Testing	4
5. Chapter 4 — Test Analysis and Design	4
6. Chapter 5 — Managing the Test Activities	5
7. Chapter 6 — Test Tools	5
8. Study Drills Tied to the Project	5

1. How to Use This Document

The ISTQB Certified Tester Foundation Level (CTFL) v4.0 exam rewards connecting syllabus concepts to real practice. QA Atelier was built so that every examinable idea has a concrete, lived example. Read each chapter section here, then find the corresponding artifact in the repository — recalling a personal example is far easier than recalling a definition.

Exam facts: 40 multiple-choice questions, closed book; pass at 26/40 (65%) in 60 minutes (+25% in a non-native language). Chapters 4 and 5 carry 20 of 40 questions and all K3 “apply” questions — that is where the points are.

2. Chapter 1 — Fundamentals of Testing

- **Test objectives vary by context.** Here they are: defect detection (negative API cases), confidence in the critical path (P0 smoke tests), and portfolio evidence of professional process.
- **The seven testing principles, each with a project example:**
 1. *Testing shows the presence of defects, not their absence* — the `body.token` failure proved our assumption about the API wrong; it proved nothing about the app being correct.
 2. *Exhaustive testing is impossible* — requirements are prioritized P0/P1/P2 instead of “test everything”.
 3. *Early testing saves time and money* — API contract tests were written before UI automation; the broken `api.yml` was caught by review before a single run.
 4. *Defects cluster together* — auth flows received the most negative test cases.
 5. *Tests wear out (pesticide paradox)* — `type: exploratory` issues exist to keep feeding new cases into the suite.
 6. *Testing is context dependent* — RWA’s cookie-based auth shaped the entire API test design; a JWT app would be tested differently.
 7. *Absence-of-errors fallacy* — green tests against a wrong contract are worthless; verify against the real server, not the documentation.
- **Test process activities:** planning (Document 1), analysis and design (test-case issues), implementation and execution (specs + CI), evaluating completion (RTM exit criteria), completion (archive).
- **Testware:** this documentation set, issues, collections, scripts, environments — all version-controlled.
- **Traceability:** the RTM links requirements to tests and supports coverage evaluation — a core K2 exam theme.

3. Chapter 2 — Testing Throughout the SDLC

- **Test levels:** Vitest/Newman suites operate at component/integration level (API contracts); Cypress/Playwright specs at system level (E2E); CI binds the levels per commit.
- **Test types:** functional (most cases), non-functional (TC-020 response-time budget), regression (scheduled weekday runs), smoke (the P0 subset).
- **Shift-left:** API testing before UI automation; contract assertions catch breaks earlier and cheaper.
- **Maintenance testing:** an RWA submodule update triggers full regression through path-filtered workflows.

4. Chapter 3 — Static Testing

- **Reviews are static testing.** Every pull request is a review event — the invalid `api.yml` indentation and the illegal `working-directory` key on a `uses:` step were found statically, before execution. Static testing finds defects early and cheaply, and finds defects that execution cannot (e.g., missing requirement coverage in the RTM).
- **Static analysis:** TypeScript compilation of spec files is automated static analysis — it catches errors before any test runs.

5. Chapter 4 — Test Analysis and Design

The heaviest exam chapter (11 of 40 questions). Each technique with its project application:

Technique	Applied in QA Atelier
Equivalence partitioning	Login credentials (valid / wrong password / unknown user); amounts (positive / zero / negative)
Boundary value analysis	Payment amount: 0.01; exactly equal to balance; balance – 0.01 (TC-014)
Decision table testing	Transaction outcomes: sufficient balance × pay/request × accept/reject combinations
State transition testing	Session states (anonymous → authenticated → logged out); transaction states (pending → accepted/rejected)
Experience-based / error guessing	The empty-string-secret CI defect came from guessing how GitHub expands missing secrets
Exploratory testing	Session notes logged as <code>type: exploratory</code> issues, feeding new catalog entries

Specification-based (black-box) cases fill the catalog; experience-based sessions keep it honest. Structure-based (white-box) coverage is out of scope for this black-box portfolio — a defensible, documented choice.

6. Chapter 5 — Managing the Test Activities

- **Test planning:** Document 1 is the plan — scope, approach, resources, risk priorities; the board is the schedule.
- **Risk-based prioritization:** P0/P1/P2 labels map to product risk; smoke on every PR, full regression on a schedule.
- **Entry criteria:** RWA healthy (sanity login curl passes), branch CI green. **Exit criteria:** all P0 automated and passing; no P0/P1 RTM row below “automated”.
- **Monitoring and control:** CI run history, uploaded artifacts (reports, screenshots, traces), board velocity.
- **Configuration management:** everything versioned — code, tests, environments, `flake.lock` pinning the toolchain itself.
- **Defect management:** type: bug-found issues with reproduction steps; severity via priority labels; lifecycle on the board.

7. Chapter 6 — Test Tools

- **Tool selection needs a rationale** (a K2 favorite): the framework decision matrix assigns each tool a purpose — Playwright for modern multi-browser E2E, Cypress for RWA flows and debugging, Selenium for legacy/Python pipelines, UIAutomator for Android, Postman/Newman for API contracts.
- **Automation benefits and risks, demonstrated honestly:** speed and repeatability in CI, versus two classic pitfalls hit in practice — over-reliance on an assumed contract (the nonexistent token) and timing flakiness (the RWA startup race). The syllabus names both.
- **Pilot before scaling:** the auth domain was automated end-to-end before expanding to transactions — the recommended tool-introduction approach.

8. Study Drills Tied to the Project

1. Recite the seven principles, each with its QA Atelier example, without looking.
2. Write EP classes and boundaries for “payment amount” in under 3 minutes (K3 pace).
3. Draw the state diagram of a money request (pending → accepted/rejected) and derive test cases from it.
4. Explain why reviewing a pull request is static testing, and name two defects it caught here.
5. State the project’s entry and exit criteria from memory, then check Document 1.